

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
TAKE HOME TASK
MODUL 12
SOLID PRINCIPLES & DESIGN PATTERNS



Disusun Oleh:

Ubaidulloh Zulkarnian (NIM: 105224024)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Laporan ini disusun untuk mendokumentasikan hasil pengerjaan tugas Take Home Task (THT) Modul 12 pada praktikum Pemrograman Berorientasi Objek. Fokus utama praktikum modul ini adalah implementasi praktis dari SOLID Principles (Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, dan Dependency Inversion Principle) serta Design Patterns (Strategy Pattern dan Dependency Injection) pada sistem pengelolaan akademik mahasiswa (SIKAD) khususnya dalam pemrosesan Kartu Rencana Studi (KRS). Pengaplikasian kelima prinsip SOLID ini bertujuan untuk meminimalkan keterikatan antar-komponen (low coupling), meningkatkan kemudahan pengembangan (high maintainability), serta mempermudah penambahan fitur baru (high extensibility) tanpa merusak fungsionalitas yang telah berjalan stabil.

II. Variabel

No	Nama Variabel	Tipe Data	Letak Kelas	Fungsi
1	nim	String	Mahasiswa	Menyimpan Nomor Induk Mahasiswa sebagai identitas unik.
2	nama	String	Mahasiswa	Menyimpan nama lengkap mahasiswa.
3	jalurMasuk	JalurMasuk	Mahasiswa	Menyimpan jalur masuk kuliah (Enum: REGULER, KARYAWAN, INTERNASIONAL, BIDIKMISI).
4	riwayatMataKuliah	List<String>	Mahasiswa	Menyimpan riwayat kode mata kuliah yang telah diselesaikan oleh mahasiswa.
5	daftarMataKuliah	List<MataKuliah>	Krs	Menyimpan kumpulan mata kuliah yang diambil mahasiswa dalam KRS berjalan.
6	kode	String	MataKuliah	Kode unik mata kuliah (misalnya: IF211).
7	nama	String	MataKuliah	Nama mata kuliah (misalnya: Pemrograman Berorientasi Objek).
8	sks	int	MataKuliah	Jumlah bobot SKS standar mata kuliah.
9	prereqKode	String	MataKuliah	Kode mata kuliah prasyarat yang harus ditempuh sebelumnya (null jika tidak ada).
10	namaLab	String	MataKuliahPraktikum	Nama laboratorium tempat pelaksanaan praktikum.
11	lokasi	String	MataKuliahLapangan	Nama lokasi fisik pelaksanaan kuliah lapangan/magang.

12	dbMock	Map<String, Krs>	MySqlKrsRepository / ...	Struktur data mock untuk mensimulasikan penyimpanan data KRS di memori.
----	--------	------------------	--------------------------	---

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Letak Kelas	Fungsi
1	Mahasiswa()	Constructor	Mahasiswa	Menginisialisasi objek mahasiswa baru dengan nim, nama, dan jalur masuk.
2	tambahRiwayat()	Method (Procedural)	Mahasiswa	Menambahkan kode mata kuliah ke dalam daftar riwayat kelulusan.
3	sudahLulus()	Method (Functional)	Mahasiswa	Memeriksa apakah mahasiswa sudah lulus mata kuliah prasyarat tertentu.
4	tambahMataKuliah()	Method (Procedural)	Krs	Menambahkan mata kuliah ke dalam draf KRS berjalan.
5	hitungTotalSks()	Method (Functional)	Krs	Menghitung total SKS di KRS dengan mendelegasikan ke strategi kalkulasi.
6	prosesKrs()	Method (Procedural)	KrsService	Memproses pendaftaran KRS, memvalidasi prasyarat, menghitung UKT, menyimpan ke database, dan mencetak PDF.
7	validatePrereq()	Method (Functional)	PrereqValidationService	Memeriksa kesesuaian riwayat kelulusan mahasiswa dengan prasyarat mata kuliah.
8	generateKrsPdf()	Method (Procedural)	KrsPdfGenerator	Mencetak visualisasi kartu rencana studi serta rincian biaya UKT ke layar.
9	calculateTotalSks()	Method (Functional)	SksCalculationStrategy	Interface untuk kalkulasi SKS berdasarkan skema kurikulum tertentu.
10	calculateUkt()	Method (Functional)	UktCalculationStrategy	Interface untuk kalkulasi tarif UKT mahasiswa berdasarkan jalur masuk.
11	save()	Method (Procedural)	KrsRepository	Interface untuk operasi penyimpanan data KRS mahasiswa.
12	alokasiAsistenLab()	Method (Procedural)	AsistenLabAllocatable	Interface untuk alokasi asisten laboratorium pada mata kuliah praktikum.

13	cekPeralatanPraktikum()	Method (Procedural)	PeralatanCheckable	Interface untuk pengecekan kelayakan peralatan lab praktikum.
----	-------------------------	---------------------	--------------------	---

IV. Dokumentasi dan Pembahasan Code

1. Dokumentasi Source Code

Di bawah ini merupakan visualisasi struktur source code dari kelas-kelas pembentuk sistem SIAKAD SOLID Modul 12. Dokumentasi ini terbagi berdasarkan kategori kelasnya:

- a. Mahasiswa.java: Menyimpan identitas mahasiswa (NIM, Nama, Jalur Masuk) beserta daftar riwayat mata kuliah yang telah diselesaikannya.

```

1  package model;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  Windsurf: Refactor | Explain
7  public class Mahasiswa {
8      private String nim;
9      private String nama;
10     private JalurMasuk jalurMasuk;
11     private List<String> riwayatMataKuliah; // List kode mata kuliah yang sudah lulus
12
13     public Mahasiswa(String nim, String nama, JalurMasuk jalurMasuk) {
14         this.nim = nim;
15         this.nama = nama;
16         this.jalurMasuk = jalurMasuk;
17         this.riwayatMataKuliah = new ArrayList<>();
18     }
19
20     Windsurf: Refactor | Explain | Generate Javadoc | X
21     public String getNim() {
22         return nim;
23     }
24
25     Windsurf: Refactor | Explain | Generate Javadoc | X
26     public String getNama() {
27         return nama;
28     }
29
30     Windsurf: Refactor | Explain | Generate Javadoc | X
31     public JalurMasuk getJalurMasuk() {
32         return jalurMasuk;
33     }
34
35     Windsurf: Refactor | Explain | Generate Javadoc | X
36     public List<String> getRiwayatMataKuliah() {
37         return riwayatMataKuliah;
38     }
39
40     Windsurf: Refactor | Explain | Generate Javadoc | X
41     public void tambahRiwayat(String kodeMataKuliah) {
42         this.riwayatMataKuliah.add(kodeMataKuliah);
43     }
44
45     Windsurf: Refactor | Explain | Generate Javadoc | X
46     public boolean sudahLulus(String kodeMataKuliah) {
47         return riwayatMataKuliah.contains(kodeMataKuliah);
48     }
49 }

```

Gambar 1. Source code Mahasiswa.java

- b. Krs.java: Menyimpan draf rencana studi semester aktif dan mendelegasikan perhitungan total kredit SKS ke SksCalculationStrategy.

```

1  package model;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import service.sks.SksCalculationStrategy;
6
7  Windsurf: Refactor | Explain
8  public class Krs {
9      private Mahasiswa mahasiswa;
10     private List<MataKuliah> daftarMataKuliah;
11
12     public Krs(Mahasiswa mahasiswa) {
13         this.mahasiswa = mahasiswa;
14         this.daftarMataKuliah = new ArrayList<>();
15     }
16
17     Windsurf: Refactor | Explain | Generate Javadoc | X
18     public Mahasiswa getMahasiswa() {
19         return mahasiswa;
20     }
21
22     Windsurf: Refactor | Explain | Generate Javadoc | X
23     public List<MataKuliah> getDaftarMataKuliah() {
24         return daftarMataKuliah;
25     }
26
27     Windsurf: Refactor | Explain | Generate Javadoc | X
28     public void tambahMataKuliah(MataKuliah mk) {
29         this.daftarMataKuliah.add(mk);
30     }
31
32     // Menggunakan Strategy Pattern untuk menghitung total SKS
33     Windsurf: Refactor | Explain | X
34     public int hitungTotalSks(SksCalculationStrategy strategy) {
35         return strategy.calculateTotalSks(this.daftarMataKuliah);
36     }
37 }

```

Gambar 2. Source code Krs.java

- c. MataKuliah.java: Kelas basis abstrak yang mendefinisikan informasi dasar mata kuliah seperti kode, nama, sks, dan kode prasyarat.

```

1 package model;
2
3 Windsurf: Refactor | Explain
4 public abstract class MataKuliah {
5     private String kode;
6     private String nama;
7     private int sks;
8     private String prereqKode; // Kode mata kuliah prasyarat (null jika tidak ada)
9
10    public MataKuliah(String kode, String nama, int sks) {
11        this(kode, nama, sks, prereqKode: null);
12    }
13
14    public MataKuliah(String kode, String nama, int sks, String prereqKode) {
15        this.kode = kode;
16        this.nama = nama;
17        this.sks = sks;
18        this.prereqKode = prereqKode;
19    }
20
21    Windsurf: Refactor | Explain | Generate Javadoc | X
22    public String getKode() {
23        return kode;
24    }
25
26    Windsurf: Refactor | Explain | Generate Javadoc | X
27    public String getNama() {
28        return nama;
29    }
30
31    Windsurf: Refactor | Explain | Generate Javadoc | X
32    public int getSks() {
33        return sks;
34    }
35
36    Windsurf: Refactor | Explain | Generate Javadoc | X
37    public String getPrereqKode() {
38        return prereqKode;
39    }
40
41    Windsurf: Refactor | Explain | Generate Javadoc | X
42    public boolean hasPrereq() {
43        return prereqKode != null && !prereqKode.isEmpty();
44    }
45
46    Windsurf: Refactor | Explain | Generate Javadoc | X
47    @Override
48    public String toString() {
49        return nama + " (" + kode + ", " + sks + " SKS)";
50    }
51 }

```

Gambar 3. Source code MataKuliah.java

- d. MataKuliahPraktikum.java: Subkelas dari MataKuliah yang merepresentasikan praktikum laboratorium dengan mengimplementasikan alokasi asisten dan cek peralatan.

```

1 package model;
2
3 Windsurf Refactor | Explain
4 public class MataKuliahPraktikum extends MataKuliah implements AsistenLabAllocatable, PeralatanCheckable {
5     private String namaLab;
6     private int jumlahAsisten;
7     private boolean peralatanSiap;
8
9     public MataKuliahPraktikum(String kode, String nama, int sks, String namaLab) {
10         super(kode, nama, sks);
11         this.namaLab = namaLab;
12         this.jumlahAsisten = 0;
13         this.peralatanSiap = false;
14     }
15
16     public MataKuliahPraktikum(String kode, String nama, int sks, String prereqCode, String namaLab) {
17         super(kode, nama, sks, prereqCode);
18         this.namaLab = namaLab;
19         this.jumlahAsisten = 0;
20         this.peralatanSiap = false;
21     }
22
23 Windsurf Refactor | Explain | Generate Javadoc | X
24 @Override
25 public void alokasiAsistenLab() {
26     this.jumlahAsisten = 2; // Simulating allocation of 2 assistants
27     System.out.println("Berhasil mengalokasikan " + jumlahAsisten + " asisten lab untuk " + getName() + " di " + namaLab);
28 }
29
30 Windsurf Refactor | Explain | Generate Javadoc | X
31 @Override
32 public void cekPeralatanPraktikum() {
33     this.peralatanSiap = true; // Simulating successful equipment check
34     System.out.println("Peralatan praktikum di " + namaLab + " untuk " + getName() + " diperiksa: SIAP");
35 }
36
37 Windsurf Refactor | Explain | Generate Javadoc | X
38 public String getNamaLab() {
39     return namaLab;
40 }
41
42 Windsurf Refactor | Explain | Generate Javadoc | X
43 public int getJumlahAsisten() {
44     return jumlahAsisten;
45 }
46
47 Windsurf Refactor | Explain | Generate Javadoc | X
48 public boolean isPeralatanSiap() {
49     return peralatanSiap;
50 }

```

Gambar 4. Source code MataKuliahPraktikum.java

- e. Subkelas Mata Kuliah Lainnya (Teori & Lapangan) dan Interface Operasi Praktikum:

```

1 package model;
2
3 Windsurf Refactor | Explain
4 public class MataKuliahTeori extends MataKuliah {
5     public MataKuliahTeori(String kode, String nama, int sks) {
6         super(kode, nama, sks);
7     }
8
9     public MataKuliahTeori(String kode, String nama, int sks, String prereqCode) {
10         super(kode, nama, sks, prereqCode);
11     }
12 }

```

Gambar 5. Source code MataKuliahTeori.java

```

1 package model;
2
3 public class MataKuliahLapangan extends MataKuliah {
4     private String lokasi;
5
6     public MataKuliahLapangan(String kode, String nama, int sks, String lokasi) {
7         super(kode, nama, sks);
8         this.lokasi = lokasi;
9     }
10
11     public MataKuliahLapangan(String kode, String nama, int sks, String prereqCode, String lokasi) {
12         super(kode, nama, sks, prereqCode);
13         this.lokasi = lokasi;
14     }
15
16     public String getLokasi() {
17         return lokasi;
18     }
19 }

```

Gambar 6. Source code MataKuliahLapangan.java

```

1 package model;
2
3 public interface AsistenLabAllocatable {
4     void alokasiAsistenLab();
5 }

```

Gambar 7. Interface AsistenLabAllocatable.java

```

1 package model;
2
3 public interface PeralatanCheckable {
4     void cekPeralatanPraktikum();
5 }
6

```

Gambar 8. Interface PeralatanCheckable.java

- f. KrsRepository.java: Interface yang menjadi kontrak abstraksi penyimpanan data KRS.

```

1 package repository;
2
3 import model.Krs;
4
5 public interface KrsRepository {
6     void save(Krs krs);
7     Krs findByNim(String nim);
8 }
9

```

Gambar 9. Interface KrsRepository.java

- g. Implementasi Repositori Concret (MySQL & Cloud NoSQL):

```

1 package repository;
2
3 import java.util.HashMap;
4 import java.util.Map;
5 import model.Krs;
6
7 public class MySqlKrsRepository implements KrsRepository {
8     private Map<String, Krs> dbMock = new HashMap<>();
9
10    @Override
11    public void save(Krs krs) {
12        dbMock.put(krs.getMahasiswa().getNim(), krs);
13        System.out.println("Data KRS mahasiswa " + krs.getMahasiswa().getNama() + " (" + krs.getMahasiswa().getNim() + ") berhasil disimpan");
14    }
15
16    @Override
17    public Krs findByNim(String nim) {
18        return dbMock.get(nim);
19    }
20 }
21

```

Gambar 10. Source code MySqlKrsRepository.java

```

1 package repository;
2
3 import java.util.HashMap;
4 import java.util.Map;
5 import model.Krs;
6
7 public class CloudNoSqlKrsRepository implements KrsRepository {
8     private Map<String, Krs> documentStore = new HashMap<>();
9
10    @Override
11    public void save(Krs krs) {
12        documentStore.put(krs.getMahasiswa().getNim(), krs);
13        System.out.println("Data KRS mahasiswa " + krs.getMahasiswa().getNama() + " (" + krs.getMahasiswa().getNim() + ") berhasil disimpan");
14    }
15
16    @Override
17    public Krs findByNim(String nim) {
18        return documentStore.get(nim);
19    }
20 }
21

```

Gambar 11. Source code CloudNoSqlKrsRepository.java

- h. KrsService.java: Orkestrator alur pendaftaran KRS mahasiswa.

```
1 package service;
2
3 import model.Krs;
4 import model.Matakuliah;
5 import repository.KrsRepository;
6 import service.pdf.KrsPdfGenerator;
7 import service.sks.SksCalculationStrategy;
8 import service.ukt.UktCalculationStrategy;
9 import service.validation.PrereqValidationService;
10
11 public class KrsService {
12     private final PrereqValidationService validator;
13     private final KrsRepository repository;
14     private final KrsPdfGenerator pdfGenerator;
15     private final SksCalculationStrategy sksCalculator;
16
17     // Dependency Injection via Constructor
18     public KrsService(
19         PrereqValidationService validator,
20         KrsRepository repository,
21         KrsPdfGenerator pdfGenerator,
22         SksCalculationStrategy sksCalculator) {
23         this.validator = validator;
24         this.repository = repository;
25         this.pdfGenerator = pdfGenerator;
26         this.sksCalculator = sksCalculator;
27     }
28
29     public void prosesKrs(Krs krs, UktCalculationStrategy uktStrategy) {
30         for (Matakuliah mk : krs.getDaftarMatakuliah()) {
31             boolean isValid = validator.validatePrereq(krs.getMahasiswa(), mk);
32             if (!isValid) {
33                 System.out.println("Gagal memproses KRS " + krs.getMahasiswa().getNama() +
34                                     " : Mata kuliah " + mk.getNama() + " tidak memenuhi prasyarat.\n");
35                 return;
36             }
37         }
38
39         double uktTagihan = uktStrategy.calculateUkt(krs.getMahasiswa());
40         repository.save(krs);
41         pdfGenerator.generateKrsPdf(krs, sksCalculator, uktTagihan);
42     }
43 }
44
```

Gambar 12. Source code KrsService.java

- i. Kelas Pendukung Bisnis (Validasi Prasyarat & Cetak Laporan):

```
1 package service.validation;
2
3 import model.Mahasiswa;
4 import model.Matakuliah;
5
6 public class PrereqValidationService {
7     public boolean validatePrereq(Mahasiswa mahasiswa, Matakuliah mk) {
8         if (!mk.hasPrereq()) {
9             return true;
10        }
11
12        String prereqKode = mk.getPrereqKode();
13        return mahasiswa.sudahLulus(prereqKode);
14    }
15 }
16
```

Gambar 13. Source code PrereqValidationService.java

```

1 package service.pdf;
2
3 import model.Krs;
4 import service.sks.SksCalculationStrategy;
5
6 public class KrsPdfGenerator {
7     public void generateKrsPdf(Krs krs, SksCalculationStrategy sksCalculator, double uktTagihan) {
8         System.out.println(x: "\n=====");
9         System.out.println(x: "KARTU RENCANA STUDI (KRS)");
10        System.out.println(x: "UNIVERSITAS TEKNOLOGI NEGERI");
11        System.out.println(x: "=====");
12        System.out.println("NIM : " + krs.getMahasiswa().getNim());
13        System.out.println("Nama Mahasiswa : " + krs.getMahasiswa().getNama());
14        System.out.println("Jalur Masuk : " + krs.getMahasiswa().getJalurMasuk());
15        System.out.println("Tagihan UKT : Rp " + String.format(format: "%.2f", uktTagihan));
16        System.out.println(x: "=====");
17        System.out.println(x: "Daftar Mata Kuliah yang Diambil:");
18
19        var listMk = krs.getDaftarMataKuliah();
20        for (int i = 0; i < listMk.size(); i++) {
21            var mk = listMk.get(i);
22            System.out.println((i + 1) + ". [" + mk.getKode() + "] " + mk.getNama() + " (" + mk.getSks() + " SKS)");
23        }
24
25        int totalSks = krs.hitungTotalSks(sksCalculator);
26        System.out.println(x: "=====");
27        System.out.println("Total SKS yang Diambil: " + totalSks + " SKS");
28        System.out.println(x: "=====");
29    }
30 }
31

```

Gambar 14. Source code KrsPdfGenerator.java

j. Strategi Perhitungan SKS (Standard vs MBKM):

```

1 package service.sks;
2
3 import java.util.List;
4 import model.MataKuliah;
5
6 public interface SksCalculationStrategy {
7     int calculateTotalSks(List<MataKuliah> daftarMataKuliah);
8 }
9

```

Gambar 15. Interface SksCalculationStrategy.java

```

1 package service.sks;
2
3 import java.util.List;
4 import model.MataKuliah;
5
6 public class StandardSksCalculator implements SksCalculationStrategy {
7     @Override
8     public int calculateTotalSks(List<MataKuliah> daftarMataKuliah) {
9         int total = 0;
10        for (MataKuliah mk : daftarMataKuliah) {
11            total += mk.getSks();
12        }
13        return total;
14    }
15 }
16

```

Gambar 16. Source code StandardSksCalculator.java

```

1 package service.sks;
2
3 import java.util.List;
4 import model.MataKuliah;
5 import model.MataKuliahLapangan;
6
7 public class MbkmSksCalculator implements SksCalculationStrategy {
8     @Override
9     public int calculateTotalSks(List<MataKuliah> daftarMataKuliah) {
10        int total = 0;
11        for (MataKuliah mk : daftarMataKuliah) {
12            if (mk instanceof MataKuliahLapangan) {
13                // Berdasarkan regulasi MBKM, kuliah lapangan/magang dikonversi setara 20 SKS
14                total += 20;
15            } else {
16                total += mk.getSks();
17            }
18        }
19        return total;
20    }
21 }
22

```

Gambar 17. Source code MbkmSksCalculator.java

k. Strategi Perhitungan UKT Jalur Masuk:

```
1 package service.ukt;
2
3 import model.Mahasiswa;
4
5 public interface UktCalculationStrategy {
6     double calculateUkt(Mahasiswa mahasiswa);
7 }
8
```

Gambar 18. Interface UktCalculationStrategy.java

```
1 package service.ukt;
2
3 import model.Mahasiswa;
4
5 public class RegularUktStrategy implements UktCalculationStrategy {
6     private static final double BASE_UKT = 5000000.0;
7
8     @Override
9     public double calculateUkt(Mahasiswa mahasiswa) {
10         // Logika penghitungan UKT reguler
11         return BASE_UKT;
12     }
13 }
14
```

Gambar 19. Source code RegularUktStrategy.java

l. Driver Utama (App.java)

App.java menginisialisasi seluruh dependensi, strategi, model mata kuliah, mahasiswa, dan menguji 4 skenario simulasi SIAKAD.

```
1 import java.util.List;
2 import model.*;
3 import repository.*;
4 import service.KrsService;
5 import service.pdf.KrsPdfGenerator;
6 import service.sks.*;
7 import service.ukt.*;
8 import service.validation.PrereqValidationService;
9
10 public class App {
11     Run IDEbug
12     public static void main(String[] args) throws Exception {
13         Matakuliah pbo = new MatakuliahTeori(kode: "IF211", nama: "Pemrograman Berorientasi Objek", sks: 3);
14         Matakuliah strukdat = new MatakuliahTeori(kode: "IF212", nama: "Struktur Data", sks: 3);
15         Matakuliah dpl = new MatakuliahTeori(kode: "IF213", nama: "Desain Perangkat Lunak", sks: 3, prereqKode: "IF211");
16         Matakuliah praktikumPbo = new MatakuliahPraktikum(kode: "IF211P", nama: "Praktikum Pemrograman Berorientasi Objek", sks: 1, namaLab: "Lab IF211");
17         Matakuliah kkn = new MatakuliahLapangan(kode: "IF400", nama: "Kuliah Kerja Nyata (KKN) Tematik", sks: 4, lokasi: "Desa Sukamaju");
18
19         PrereqValidationService validator = new PrereqValidationService();
20         KrsPdfGenerator pdfGenerator = new KrsPdfGenerator();
21
22         List<Matakuliah> daftarSemuaMatakuliah = List.of(pbo, strukdat, dpl, praktikumPbo, kkn);
23         for (Matakuliah mk : daftarSemuaMatakuliah) {
24             if (mk instanceof AsistenLabAllocatable) {
25                 ((AsistenLabAllocatable) mk).alokasiAsistenLab();
26             }
27             if (mk instanceof PeralatanCheckable) {
28                 ((PeralatanCheckable) mk).cekPeralatanPraktikum();
29             }
30         }
31         System.out.println();
32
33         Mahasiswa budi = new Mahasiswa(nim: "105224001", nama: "Budi Santoso", JalurMasuk.REGULER);
34         budi.tambahRiwayat(kodeMatakuliah: "IF211");
35
36         Mahasiswa alice = new Mahasiswa(nim: "105224002", nama: "Alice Smith", JalurMasuk.INTERNASIONAL);
37
38         Mahasiswa charlie = new Mahasiswa(nim: "105224003", nama: "Charlie", JalurMasuk.BIDIKMISI);
39         charlie.tambahRiwayat(kodeMatakuliah: "IF211");
40
41         Mahasiswa daniel = new Mahasiswa(nim: "105224004", nama: "Daniel Siregar", JalurMasuk.KARYAWAN);
42
43         UktCalculationStrategy regulerUkt = new RegularUktStrategy();
44         UktCalculationStrategy internasionalUkt = new InternasionalUktStrategy();
45         UktCalculationStrategy bidikmisiUkt = new BidikmisiUktStrategy();
46         UktCalculationStrategy karyawanUkt = new KaryawanUktStrategy();
47
48         SksCalculationStrategy standardSks = new StandardSksCalculator();
49     }
50 }
```

Gambar 20. Source code driver utama App.java

2. Analisis Penerapan SOLID Principles pada Codebase

Berikut merupakan analisis mendalam terhadap penerapan kelima prinsip SOLID pada kode program SIAKAD yang telah diimplementasikan:

a. Single Responsibility Principle (SRP)

Prinsip SRP menetapkan bahwa setiap kelas hanya boleh bertanggung jawab atas satu fungsionalitas tunggal. Pemisahan tanggung jawab ini terlihat sangat jelas pada arsitektur kelas layanan (service):

- KrsService hanya bertanggung jawab mengoordinasikan langkah-langkah pengisian KRS mahasiswa.
- PrereqValidationService khusus menangani validasi apakah mahasiswa berhak mengambil mata kuliah prasyarat.
- KrsPdfGenerator khusus memformat dan mencetak hasil KRS ke bentuk PDF/Layar.

Jika format tampilan PDF berubah, hanya kelas KrsPdfGenerator yang dimodifikasi. Logika bisnis pendaftaran di KrsService tetap aman.

b. Open/Closed Principle (OCP)

Prinsip OCP mengharuskan perangkat lunak terbuka untuk perluasan (open for extension) tetapi tertutup untuk modifikasi (closed for modification). Pola ini diterapkan melalui Strategy Pattern pada penghitungan UKT dan SKS:

- Perhitungan UKT menggunakan interface UktCalculationStrategy. Ketika ada penambahan tarif untuk jalur masuk baru (misalnya Jalur Beasiswa Khusus), kita tidak perlu mengubah kelas KrsService, melainkan cukup menulis kelas baru seperti BidikmisiUktStrategy.
- Perhitungan total SKS di dalam KRS didelegasikan kepada SksCalculationStrategy. Hal ini memungkinkan kita menghitung SKS secara standar maupun konversi SKS MBKM (20 SKS untuk kuliah lapangan) secara independen.

c. Liskov Substitution Principle (LSP)

LSP menuntut bahwa objek superkelas harus dapat digantikan oleh objek subkelasnya tanpa merusak integritas program. Pada kode program, kelas basis abstrak MataKuliah didefinisikan secara umum. Subkelasnya yaitu MataKuliahTeori, MataKuliahPraktikum, dan MataKuliahLapangan mewarisi struktur kelas basis dengan benar. Semua subkelas ini dapat disimpan dalam list polimorfis (List<MataKuliah>) di dalam kelas KRS dan diproses tanpa menghasilkan crash atau kesalahan tipe objek.

d. Interface Segregation Principle (ISP)

ISP menyatakan bahwa kelas tidak boleh dipaksa mengimplementasikan metode yang tidak digunakannya. Jika kita menyatukan metode khusus laboratorium (seperti alokasi asisten laboratorium dan pengecekan peralatan praktikum) langsung ke dalam kelas abstrak MataKuliah, subkelas MataKuliahTeori dan MataKuliahLapangan terpaksa mengimplementasikannya dan memicu pelemparan error. Maka dari itu, fungsionalitas tersebut dipecah ke dalam interface terpisah: AsistenLabAllocatable dan PeralatanCheckable. Hanya kelas

MataKuliahPraktikum yang mengimplementasikan antarmuka ini. Kelas teori dan lapangan tetap bersih dari metode yang tidak relevan.

e. Dependency Inversion Principle (DIP)

DIP menegaskan bahwa modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah, melainkan harus bergantung pada abstraksinya. Kelas layanan utama KrsService (tingkat tinggi) sama sekali tidak mengimpor atau memanggil instansiasi langsung dari MySqlKrsRepository atau CloudNoSqlKrsRepository (tingkat rendah). KrsService hanya bergantung pada interface KrsRepository. Implementasi konkret disuntikkan secara dinamis melalui konstruktor (Constructor Injection) saat inisialisasi di kelas driver App. Hal ini mempermudah transisi tipe penyimpanan basis data dari relasional ke non-relasional tanpa mengubah baris kode logika bisnis sedikit pun.

3. Hasil output Program

Eksekusi pengujian program utama dilakukan dengan mensimulasikan empat skenario utama mahasiswa:

- Skenario A (Budi): Mahasiswa reguler berhasil mendaftarkan mata kuliah prasyarat karena ia telah lulus PBO (IF211), data berhasil disimpan ke SQL Database.
- Skenario B (Alice): Mahasiswa internasional gagal mendaftarkan mata kuliah DPL karena tidak memenuhi prasyarat kelulusan PBO, menyebabkan pembatalan KRS secara aman.
- Skenario C (Charlie): Menunjukkan perbedaan kalkulasi SKS reguler (terhitung 7 SKS) dan skema konversi MBKM (terhitung 23 SKS karena mata kuliah lapangan dikonversi setara 20 SKS), data disimpan ke Cloud NoSQL dalam bentuk dokumen terformat JSON.
- Skenario D (Daniel): Mahasiswa karyawan berhasil melakukan registrasi KRS dengan pengenalan tarif UKT khusus kelas karyawan.

```
PS E:\KuliahPraktikum\PROJ\modul_12\IF211> java -cp .\KuliahPraktikum\PROJ\modul_12\IF211\1 & 4. Program File: C:\Users\jani> .\main preview -> OK <Close> <Detail> <Print> <PrintMessages> -> OK
KULIAH/praktikum PROJ\modul_12\IF211\1> app
Berhasil menglokasikan 2 asisten lab untuk Praktikum Pemrograman Berorientasi Objek di Lab IPA
Peraturan praktikum di Lab IPA untuk Praktikum Pemrograman Berorientasi Objek diperiksa: SIM
Data KRS mahasiswa Budi Santoso (185224001) berhasil disimpan ke database MySQL.

KARTU BUDHAAN STUDE (KRS)
UNIVERSITAS TEKNOLOGI NEGARA

NPM : 185224001
Nama Mahasiswa : Budi Santoso
Jalur Masuk : REGULER
Tagihan UKT : Rp 5.400.000,00

Daftar Mata Kuliah yang Diambil:
1. [IF211] Pemrograman Berorientasi objek (3 SKS)
2. [IF213] Desain Perangkat Lunak (3 SKS)
3. [IF214] Praktikum Pemrograman Berorientasi objek (1 SKS)
Total SKS yang Diambil: 7 SKS

Gagal memproses KRS Alice Smith: Mata kuliah Desain Perangkat Lunak tidak memenuhi prasyarat.
Data KRS mahasiswa Charlie (185224003) berhasil disimpan ke database Cloud NoSQL.

KARTU BUDHAAN STUDE (KRS)
UNIVERSITAS TEKNOLOGI NEGARA

NPM : 185224003
Nama Mahasiswa : Charlie
Jalur Masuk : INTERNASIONAL
Tagihan UKT : Rp 9,00

Daftar Mata Kuliah yang Diambil:
1. [IF212] Struktur Data (3 SKS)
2. [IS400] Kuliah Kerja Nyata (KKN) Tematik (4 SKS)
Total SKS yang Diambil: 7 SKS

Data KRS mahasiswa Charlie (185224003) berhasil disimpan ke database Cloud NoSQL.

KARTU BUDHAAN STUDE (KRS)
UNIVERSITAS TEKNOLOGI NEGARA

NPM : 185224003
Nama Mahasiswa : Charlie
Jalur Masuk : INTERNASIONAL
Tagihan UKT : Rp 9,00
```

Gambar 21. Output Konsol Terminal dari Keempat Skenario Pengujian

V. Kesimpulan

Penerapan SOLID Principles dan Design Patterns (Strategy Pattern & Dependency Injection) pada pengembangan sistem SIAKAD berhasil membuktikan efisiensinya dalam menghasilkan perangkat lunak yang andal. Pemisahan tanggung jawab (SRP) mencegah

ketergantungan erat antar-kelas. Struktur interface yang didefinisikan dengan baik (ISP & OCP) mempermudah pengembangan skema bisnis kalkulasi baru di masa depan tanpa memodifikasi kode inti. Hubungan dependensi yang dibalik ke arah abstraksi (DIP) memungkinkan perubahan teknologi modul tingkat bawah seperti database tanpa mengganggu kestabilan logika bisnis tingkat atas. Adopsi prinsip-prinsip ini sangat disarankan untuk pengembangan perangkat lunak skala menengah hingga besar.

VI. Daftar Pustaka

SOLID Principles with Real Life Examples. (2026, April 16). Retrieved from
geeksforgeeks: <https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/>